Concurrent Transactional KV Store with WAL and Two-Phase Locking

Advanced Database Management — Computer Science (A & B), 4th Semester

Project 02 Manual / *Languages: C or C++*

1 Problem Statement

You will build a transactional key-value store that survives crashes, accepts many concurrent clients over the network, and provides real isolation guarantees between their transactions. This is the same family of project as the simpler WAL project that some other classes assign — but it is harder in three specific ways. First, the server is multi-client: it speaks a TCP protocol and many clients can hold open transactions simultaneously. Second, those transactions are properly isolated using strict two-phase locking with deadlock detection, so two clients editing the same keys see consistent results without ever observing each other's uncommitted state. Third, the durability machinery is fast: instead of paying the cost of `fsync` on every commit, the server batches commits together and pays one `fsync` for many transactions at once, while still guaranteeing that no acknowledged commit is ever lost.

The defining property of the system is the combination of *durability* and *serializability*. Durability means: once the server has reported `COMMIT OK` to a client, the data from that transaction must survive any subsequent crash, even one that happens microseconds after the acknowledgement. Serializability means: even when many clients run interleaved transactions concurrently, the visible outcome is always equivalent to *some* serial execution of those transactions one after another. Either property alone is straightforward; achieving both at once, with reasonable performance, is what real database engines do, and what this project asks you to do.

You will use three classical techniques. The first is write-ahead logging (WAL): before any change becomes visible, a record describing the change is appended to a binary log on disk. The second is strict two-phase locking (S2PL): each transaction acquires a shared or exclusive lock on every key it touches, holds those locks until commit or abort, and never re-acquires after releasing. The third is group commit: when many transactions are committing at the same time, the server batches their commit records into a single `fsync` call, amortizing the cost. Each technique is independently well-understood and well-documented; the work of this project is implementing all three correctly and getting them to interact without bugs.

You are not building a full SQL database. There are no tables, no schemas, no joins, no query planner, and no indexes. The data model is the same flat key-value store as in the simpler WAL project. What is added compared to the simpler project is the entire concurrency layer — the lock manager, the per-client transaction state, the deadlock detector, the group commit coordinator — plus a chaos test that runs many adversarial concurrent workloads and verifies serializability after each. Recovery itself is still redo-only and still fits in two pages of code, because at most one transaction is ever in a "committed but not yet applied" state at any moment, and aborts release locks without leaving any trace in the log that needs to be undone.

This project is one of the harder ones in Project 02. The locking bugs you will hit are the canonical kind of bug that distinguishes database engineers from application engineers: they are timing-dependent, they reproduce one in a hundred runs, and they manifest as "the answer is wrong" rather than as a crash. Plan time for testing under load and for reading your own lock manager carefully. The chaos test in Phase 3 is calibrated to surface these bugs deterministically, so a passing chaos test is meaningful evidence that your implementation is correct.

2 What the Final Product Looks Like

When your project is complete, you can start the server, point several clients at it, and watch them run interleaved transactions correctly.

```
$ make
$ ./kvdb-server --data ./db --port 7000 --group-commit-ms 5
[server] starting up...
```

```
[server] reading data file: 0 records
[server] replaying log: 0 records
[server] recovery complete
[server] lock manager initialized
[server] group commit interval: 5ms
[server] listening on port 7000

# in another terminal: client A
$ ./kvdb-cli localhost 7000
connected to kvdb (session 1)

A> BEGIN ISOLATION READ_COMMITTED;
txn 1 started.

A> PUT alice 100;
OK (locked alice exclusive, pending in txn 1).

A> PUT bob 50;
OK (locked bob exclusive, pending in txn 1).

A> COMMIT;
txn 1 committed (LSN 4, group of 1, fsync 1.2ms).

# in a third terminal: client B
$ ./kvdb-cli localhost 7000
connected to kvdb (session 2)

B> BEGIN ISOLATION REPEATABLE_READ;
txn 2 started.

B> GET alice;
100 (locked alice shared, in txn 2)

B> GET bob;
50 (locked bob shared, in txn 2)

# meanwhile in client A: another transaction tries to write alice
A> BEGIN;
txn 3 started.

A> PUT alice 999;
[blocked: waiting for shared lock on alice held by txn 2 (timeout 3s)]
```

A's transaction 3 blocks because B's transaction 2 still holds a shared lock on `alice` (B is in repeatable read mode, so the lock is held until commit). A is correctly forced to wait. When B commits, A's lock acquisition unblocks:

```
B> COMMIT;
txn 2 committed (LSN 5, group of 1, fsync 1.1ms).

# in client A, the blocked PUT now completes
A> [unblocked after 412ms]
OK (locked alice exclusive, pending in txn 3).

A> COMMIT;
txn 3 committed (LSN 6, group of 1, fsync 1.0ms).

A> \stats
session:           1
active txn:        none
locks held:        0
total commits:     2
deadlock aborts:   0
```

The server can also kill itself to demonstrate recovery, just like the simpler WAL project:

```
A> CRASH;
[server process terminates immediately]

$ ./kvdb-server --data ./db --port 7000
[server] starting up...
[server] reading data file: 0 records
[server] replaying log: 6 records
  LSN 1-4: txn 1 COMMIT (alice=100, bob=50) -> applied
  LSN 5:   txn 2 (read-only) -> nothing to apply
  LSN 6:   txn 3 COMMIT (alice=999) -> applied
[server] recovery complete: 2 committed transactions applied,
         0 partial transactions ignored
[server] listening on port 7000
```

The two things to notice: (1) concurrent transactions on the same keys block each other rather than corrupting the data, and (2) crash recovery still works correctly even with multi-client history in the log.

3 Architectural Overview

The server has six components. Keep them separated.

```
                [ TCP clients ]
                       |
                       v
              +-----------------+
              |  Connection     |  one thread per client,
              |  handlers       |  protocol parsing
              +--------+--------+
                       |
                       v
              +-----------------+
              |  Transaction    |  per-client txn state,
              |  manager        |  pending buffer, isolation
              +-----+----+------+
                    |    |
              +-----+    +------+
              |                 |
              v                 v
        +------------+    +-------------+
        |  Lock      |    |  Store      |  shared committed state
        |  manager   |    |  (in-mem)   |  protected by lock mgr
        +------+-----+    +------+------+
              |                 |
              v                 v
        +-------------------------------+
        |  WAL writer + group commit    |  appends, fsyncs
        +---------------+---------------+
                       |
                       v
                 [ wal.log file ]
```

The connection handlers are the network-facing layer: one thread per client, reading and writing the protocol. The transaction manager owns each client's currently-active transaction (if any), including its pending buffer and its isolation level. The lock manager arbitrates access to keys: every `GET`/`PUT`/`DELETE` acquires a lock first, blocking if necessary. The store is the in-memory committed state, protected by the locks. The WAL writer + group commit is the durability layer, which the rest of the project goes through to actually persist anything.

The key invariant is that **shared mutable state is touched only through the lock manager's permission**. There is no direct path from a client thread to the store that bypasses the locks. This is what makes serializability provable rather than just hoped-for.

4 The Wire Protocol

Plain text, line-based, one command per line, terminated by a newline. Same style as the other Project 02 manuals.

4.1 Session Setup

A client connects via TCP. The server immediately replies with a one-line greeting that includes the assigned session id:

```
KVDB ready (session 17)
```

No handshake or authentication. The client can then issue commands.

4.2 Transaction Commands

- `BEGIN [ISOLATION level];` — Start a new transaction. `level` is `READ_COMMITTED` or `REPEATABLE_READ`; if omitted, defaults to `REPEATABLE_READ`. Fails if a transaction is already active in this session.
- `PUT` *key value*`;` — Store a key-value pair. Acquires an exclusive lock on *key*, blocking if necessary. The change is staged in the transaction's pending buffer and logged as a `PUT` log record.
- `DELETE` *key*`;` — Same as `PUT`, but stages a tombstone.
- `GET` *key*`;` — Return the value for a key. Lock acquisition depends on isolation level (Section 6).
- `COMMIT;` — Commit the current transaction. The server writes a `COMMIT` log record, joins the next group commit batch, waits for the batched `fsync` to complete, applies the pending buffer to the store, releases all locks held by this transaction, and replies `OK`.
- `ABORT;` — Discard the pending buffer, write an `ABORT` log record (no fsync needed), release all locks, reply `OK`.

4.3 Diagnostic Commands

- `\stats` — Per-session statistics: session id, whether a transaction is active, number of locks currently held, total commits, total deadlock aborts.
- `\locks` — Print the global lock table state: every locked key, the lock mode, the holder session ids, and the wait queue if any. Used for debugging.
- `CRASH;` — For testing only. The server process terminates immediately via `_exit(1)`.
- `QUIT;` — Clean disconnect. Any active transaction is aborted before the connection closes.

5 The Lock Manager

This is the heart of the project. The lock manager arbitrates access to keys: every read or write of a key by a transaction must first acquire a lock on that key from the lock manager, and the lock manager guarantees that no two conflicting locks are held simultaneously.

5.1 Lock Modes

There are two lock modes:

- **Shared (S)**: held by readers. Multiple shared locks on the same key may coexist. A shared lock blocks new exclusive lock requests.
- **Exclusive (X)**: held by writers. An exclusive lock excludes everything: no other shared or exclusive lock on the same key may coexist.

The compatibility matrix is the standard one: S is compatible with S, everything else is incompatible.

5.2 The Lock Table

The lock manager owns a hash map from key to a `LockEntry` struct. The struct contains:

- The current lock mode (S, X, or unlocked).
- The set of holder transaction ids (one for X mode, one or more for S mode).
- A FIFO queue of waiters: pairs of (transaction id, requested mode, condition variable to signal when granted).

The lock table itself is protected by a single global mutex. This is intentionally simple: a per-key mutex would be faster but harder to get right, and the project's data sizes are small enough that the global

mutex is not the bottleneck. Document this choice in your design document.

5.3 Acquiring a Lock

To acquire a lock on a key in a given mode for a given transaction:

1. Take the lock table mutex.
2. Look up the `LockEntry` for the key (create it if it does not exist).
3. If this transaction already holds a compatible lock, return immediately (lock upgrades from S to X are also allowed if no other transaction holds an S lock; otherwise the upgrade joins the wait queue).
4. If the requested mode is compatible with the current lock state *and* the wait queue is empty, grant the lock immediately, add this transaction to the holders, and return.
5. Otherwise, append (this txn, requested mode, a fresh condition variable) to the wait queue, then wait on the condition variable. Wake-ups are triggered by other transactions releasing their locks (Section 5.4) or by the deadlock detector (Section 5.5). The wait drops the lock table mutex while it sleeps and re-acquires it on wake-up.
6. On wake-up, check whether the lock is now grantable. If yes, remove this entry from the wait queue, add to holders, and return. If the wake-up was due to a deadlock detector kill, return a deadlock error.

The wait-queue-empty check in step 4 prevents writer starvation: if step 4 only required compatibility, a stream of new shared locks could indefinitely starve a waiting exclusive lock. By requiring that the queue be empty, new shared locks must also wait if anyone is already waiting. This is a standard FIFO fairness policy.

5.4 Releasing Locks

A transaction releases all of its locks at commit or abort, in one operation. To release:

1. Take the lock table mutex.
2. For each key the transaction holds a lock on, remove this transaction from the holders. If the holders set is now empty, the lock is fully released.
3. For each fully-released lock, look at the head of the wait queue. If the head's requested mode is compatible with the current state (which is now "unlocked" for that key), grant it: remove from the wait queue, add to holders, and signal the head's condition variable. Repeat for the next head if it is also compatible. (For example, a release after an X lock might let a string of waiting S lock requests all proceed in one go.)
4. Release the lock table mutex.

5.5 Deadlock Detection

With concurrent transactions and locks, deadlocks are inevitable. Two transactions might each hold one lock and wait for the other. The lock manager must detect this and break the deadlock by aborting one of the transactions.

For this project, use a simple timeout-based detection: when a transaction has been waiting for a lock for longer than a configurable threshold (default: 3 seconds), assume it is deadlocked and abort it. This is not as elegant as cycle detection in the wait-for graph, but it is dramatically simpler to implement and is adequate for the chaos test.

The mechanism: when a transaction enters the wait queue, it records the current time. A background thread (or, equivalently, a timeout passed to the condition variable's `wait_for`) wakes up after the threshold elapses. On wake-up, if the transaction is still waiting (its lock was not granted in the meantime), the thread sets a "deadlocked" flag on the transaction's state and removes it from the wait queue, then signals the condition variable. The acquiring code returns a deadlock error to the caller, which translates to an automatic abort of the entire transaction (releasing any other locks it had acquired earlier). The client receives an error on the operation that caused the abort; it must restart the transaction.

The deadlock detector's threshold is a tunable parameter that affects benchmark numbers. A short threshold causes spurious aborts on heavy contention; a long threshold causes long stalls under real

deadlock. Document your chosen value and the reasoning.

5.6 Lock Upgrades

A transaction that already holds an S lock and now wants to write the same key needs an upgrade from S to X. Lock upgrades are notoriously tricky because they create a built-in deadlock pattern: two transactions holding S on the same key can each request an upgrade to X, and neither can proceed without the other releasing.

The simple, correct rule for this project: an upgrade is only granted if this transaction is the *only* holder of the S lock. If any other transaction also holds the S lock, the upgrade joins the wait queue (and may eventually be killed by the deadlock detector if the other holder does not release in time). This trades performance for correctness, and the chaos test will exercise the upgrade path enough to catch bugs.

6 Isolation Levels

The two supported levels are `READ_COMMITTED` and `REPEATABLE_READ`. They differ in how long shared locks are held.

6.1 REPEATABLE_READ (default)

Strict two-phase locking. Every `GET` acquires a shared lock and *holds it until commit or abort*. Every `PUT` or `DELETE` acquires an exclusive lock and holds it until commit or abort. No lock is ever released early. This guarantees that within one transaction, repeated reads of the same key always return the same value (because the lock prevents anyone else from writing it), and that no other transaction can observe this transaction's intermediate state (because the X locks block them). Strict two-phase locking provides serializability for this project's workload.

6.2 READ_COMMITTED

A weaker level. `GET` acquires a shared lock, reads, and *releases the lock immediately*. The transaction does not hold any lock on the read key after the read completes. `PUT`/`DELETE` still acquire exclusive locks and hold them until commit (this is non-negotiable; without it, dirty writes become possible). The effect is that a transaction may see different values for the same key on two separate reads (a "non-repeatable read" anomaly), but it never sees uncommitted data from another transaction (the dirty-read anomaly is prevented because writers still hold X locks until commit, and readers wait on those X locks).

This level is faster than `REPEATABLE_READ` under read-heavy workloads because read locks are released early, allowing more concurrency. The chaos test in Phase 3 includes a scenario that demonstrates the non-repeatable read anomaly, so students can see the difference between the two levels concretely.

7 The WAL and Group Commit

The log file format is the same as in the simpler WAL project, with one addition: a `txn_id` field per record so that recovery can group records by transaction. The format is described in Section 8 below. What is new in this project is **group commit**: many transactions committing concurrently are batched into a single `fsync`.

7.1 The Naive Approach (and Why It Is Slow)

The simplest correct approach is for every committing transaction to write its `COMMIT` log record, call `fsync`, and only then proceed. This is what the simpler WAL project does. The problem is that `fsync` on a typical SSD takes 1–5 milliseconds, regardless of how much data was in the request. With one transaction at a time, you cap out at a few hundred commits per second. With 100 concurrent clients all committing at once, you still cap out at a few hundred commits per second total — the `fsync` calls serialize.

7.2 Group Commit

Group commit fixes this by amortizing one `fsync` over many commits. The design is:

- There is a single "group commit" background thread.
- When a client transaction wants to commit, it appends its `COMMIT` log record to the in-memory log buffer (under a mutex), records the LSN of its commit record, then waits on a condition variable.
- The group commit thread runs on a fixed interval (e.g., every 5 milliseconds, configurable via `-group-commit-ms`). When it wakes up, it takes the mutex, flushes the in-memory log buffer to

the log file, calls `fsync`, records the `flushed_lsn` (the highest LSN now durably on disk), and signals all waiting clients.
- Each waiting client wakes up, checks whether `flushed_lsn` $\geq$ `my_commit_lsn`, and if so, proceeds to apply its pending buffer to the store, release locks, and reply OK to the user. If the wake-up was spurious (the flush did not yet cover this transaction), the client goes back to waiting.

The result: N concurrent commits within a 5 ms window pay one `fsync` between them, reducing per-commit overhead from one `fsync` per transaction to roughly one `fsync` divided by N. Under high concurrency, this is the difference between hundreds and tens of thousands of commits per second.

7.3 The Tradeoff

Group commit adds a tiny amount of latency to every commit (up to `group-commit-ms`, the worst-case wait until the next flush). This is universally worth it under concurrent load. Under single-client load, it is pure latency overhead with no benefit, which is why some real systems make it tunable (PostgreSQL has `commit_delay`; MySQL has `innodb_flush_log_at_trx_commit`). In your benchmark you will measure both regimes.

7.4 The Critical Ordering Rule

Group commit does not relax the WAL rule. Specifically: **a committing transaction must not apply its pending buffer to the store until after the group commit `fsync` that covers its commit record has completed**. If a client applied its pending buffer before the `fsync`, then crashed before the `fsync`, the store would briefly reflect uncommitted data; if some other transaction read from it (or worse, was committed depending on it), the system would have lost durability. The condition variable wait above is what enforces this ordering. Test it carefully.

8 The File Formats

Both the log file and the data file are flat binary files. All integers are little-endian. Strings are stored as a 2-byte length followed by that many bytes. Records are CRC32-checksummed; a checksum mismatch during recovery means corruption and recovery stops at that point.

8.1 Log Record Format

```
Offset  Size   Field            Description
------  -----  ---------------  ----------------------------------
  0      4     crc32            CRC32 of bytes from offset 4 to end
  4      8     lsn              uint64, monotonically increasing
 12      4     txn_id           uint32, server-assigned
 16      1     type             1=BEGIN, 2=PUT, 3=DELETE,
                                4=COMMIT, 5=ABORT
 17      ...   payload          format depends on type
```

Payloads: BEGIN/COMMIT/ABORT have empty payloads (17 bytes total). PUT has key (2-byte length + bytes) and value (2-byte length + bytes). DELETE has key only. The `txn_id` is now 4 bytes (instead of 2 in the single-client version) because many concurrent clients can each have their own active transaction.

8.2 Data File Format

Same as in the simpler WAL project: a magic header, a version, a `checkpoint_lsn`, a record count, and the entries. Written via temp-and-rename for atomicity.

9 Recovery

Recovery is performed once at startup, before the server begins accepting connections.

1. Initialize an empty store (in Phase 3, read the data file snapshot first).
2. Open the log file. If empty or absent, recovery is done.
3. Read records from the beginning, verifying each CRC. Stop at the first CRC failure.
4. Group records by `txn_id`. Unlike the simpler WAL project, transactions may interleave in the log: a single transaction's records are no longer contiguous, because multiple clients commit concurrently. You must hold per-`txn_id` state during the scan: a list of staged operations that you append to

as PUT/DELETE records arrive, finalized as committed when a COMMIT record for that txn_id arrives, or discarded when an ABORT (or end-of-log without a COMMIT) is reached.
5. After the scan, apply each committed transaction's operations to the store *in commit-LSN order* — the order of their COMMIT records, not their BEGIN records. This matters: two transactions A and B whose ranges interleave might commit in order B-then-A, and recovery must apply them in that order to match the runtime behavior. The lock manager during runtime guarantees that interleaved transactions touching the same keys cannot have conflicting writes, so commit-order replay produces the same final state.
6. Print a summary.

The interleaving requirement in step 4 is the one substantive way recovery differs from the single-client project. It is also the easiest place to write a subtle bug. Test it.

10 Phase 1 — Multi-Client Server with In-Memory Store

The goal of Phase 1 is a working TCP server that handles many concurrent clients with isolated transactions, but with no persistence at all. Crashes lose everything. The lock manager and the transaction manager are the focus.

Build:

- A TCP server on the configured port. One thread per accepted client (`pthread_create` or `std::thread`). The accept loop runs in the main thread.
- The protocol parser: line-based, dispatches commands to handlers.
- The lock manager from Section 5, including FIFO wait queue, lock upgrades, and timeout-based deadlock detection.
- The transaction manager: per-session state holding the active transaction (if any), its txn_id, its isolation level, its pending buffer, and the set of locks it holds.
- Handlers for `BEGIN` (with isolation level), `PUT`, `DELETE`, `GET`, `COMMIT` (no logging yet — just merge pending into store and release locks), `ABORT`, `\stats`, `\locks`, `QUIT`.
- The main store: a hash map of committed data, accessed only after acquiring the appropriate lock from the lock manager.
- A `kvdb-cli` client.

At the end of Phase 1 you should be able to start the server, connect two clients, and watch them interleave transactions correctly. Two clients writing the same key force one to wait for the other. Two clients writing different keys proceed in parallel. A deadlock is detected and one of the transactions is aborted within the timeout.

10.1 Phase 1 Manual Tests

Verify each of these by hand with two terminals:

1. Two clients reading the same key concurrently: both succeed without blocking.
2. One client reading, another writing the same key: the writer waits.
3. Two clients writing the same key: the second waits until the first commits.
4. A 2-cycle deadlock (A locks x then waits for y; B locks y then waits for x): one transaction is aborted within the deadlock timeout; the other proceeds.

11 Phase 2 — WAL, Group Commit, and Recovery

The goal of Phase 2 is to add the durability layer: WAL writes on every operation, group commit at COMMIT time, and crash recovery on startup.

Build:

- The WAL writer: append records to a binary file using the format in Section 8.1. The writer is shared by all client threads, protected by a mutex.
- Logging integrated into the operation handlers: BEGIN logs a BEGIN record before responding OK; PUT logs a PUT record *before* inserting into the pending buffer; DELETE same; COMMIT logs a COMMIT record but does not call fsync directly (group commit handles it); ABORT logs an ABORT

record (no fsync needed).
- The group commit thread (Section 7.2): a background thread that wakes every `group-commit-ms` milliseconds, flushes the WAL buffer, calls `fsync`, and signals all waiting committers.
- The COMMIT handler waits for the group commit thread to advance `flushed_lsn` past its commit record's LSN, then applies its pending buffer to the store and releases its locks.
- Recovery on startup (Section 9), correctly handling interleaved transaction records.

At the end of Phase 2, the example session in Section 2 works end to end including the crash and restart. Multi-client crash tests — two clients committing simultaneously, then the server is killed — recover correctly with both committed transactions visible.

12 Phase 3 — Checkpointing, Chaos Test, and Benchmark

12.1 Checkpointing

Add a `CHECKPOINT;` command. When invoked, it refuses if any transaction is active on any session, then serializes the current store to a temporary data file using the format in Section 8.2, fsyncs, renames into place, and truncates the log to remove records whose LSN $\leq$ `checkpoint_lsn`.

Update recovery: read the data file first, then replay the log records that come after.

12.2 The Chaos Test

Write a chaos test driver that connects many clients to a running server and runs adversarial concurrent workloads. After each scenario, verify the resulting state against an independent reference computation. The required scenarios:

1. **Concurrent counter:** 8 clients each run 1000 transactions, each of which reads a counter key, increments it, and writes it back. After all clients finish, the counter must equal exactly 8000. Anything else is a serializability bug. (This is the canonical lost-update test.)
2. **Read-modify-write under contention:** 4 clients share 10 keys. Each client runs 500 transactions, each of which reads two random keys, computes their sum, and writes the sum back to a third random key. The reference computation runs the same operations sequentially in the same order and produces a known final state; the actual final state must match. (Tests that 2PL with deadlock detection produces correct results even when transactions abort and retry.)
3. **Reader-writer contention:** 16 clients, 4 of which only write a fixed key in tight transactions, the other 12 of which only read it. After 30 seconds, verify that no reader ever observed a value that no writer wrote (no torn read), and that the final value is one of the values the writers wrote.
4. **Mixed isolation levels:** demonstrate the non-repeatable read anomaly under `READ_COMMITTED` and its absence under `REPEATABLE_READ`. The test runs a long-lived transaction in each mode that reads the same key twice with a sleep in between; a concurrent fast transaction modifies the key during the sleep. Under `READ_COMMITTED` the two reads return different values; under `REPEATABLE_READ` they return the same value. Both behaviors must be observable.
5. **Crash under load:** 8 clients each run 500 random-mix transactions. After 2 seconds of running, kill the server with the `CRASH` command. Restart. Verify that every transaction whose client received `COMMIT OK` is reflected in the recovered state, and that no transaction whose commit was not acknowledged left any trace.
6. **Deadlock storm:** 16 clients each run transactions that touch random pairs of keys in random order. After 5 seconds, verify that the server is still responsive (deadlock detection successfully cleared every cycle), no transaction has been blocked for longer than 2x the deadlock timeout, and the final state is consistent with some valid serial order of the committed transactions.

The chaos test is the single most important deliverable of Phase 3. Every scenario must pass. A submission that fails any scenario will receive a reduced grade for Phase 3 proportional to the number of failures.

12.3 Benchmark

Write a benchmark program that measures throughput and latency under varying conditions:

1. **Single-client commits:** 1 client, 10,000 sequential single-PUT transactions. Report transactions

per second.

2. **Concurrent commits without group commit:** 32 clients, each running 1000 single-PUT transactions, with `-group-commit-ms 0` (effectively per-commit fsync). Report aggregate transactions per second.
3. **Concurrent commits with group commit:** 32 clients, same workload, with `-group-commit-ms 5`. Report aggregate transactions per second. Expect a 10–50x improvement over the previous scenario.
4. **Read-mostly workload:** 32 clients running 95% reads, 5% writes against a 1000-key working set. Run with both `READ_COMMITTED` and `REPEATABLE_READ`. Report and discuss the difference.
5. **Recovery time:** create a database with 100,000 commits and no checkpoint, kill the server, restart, measure recovery time. Then with a checkpoint after 100,000 commits, measure again.

The first three scenarios together demonstrate the value of group commit (the single biggest performance win in the project). The fourth demonstrates the cost of holding read locks longer. The fifth demonstrates the value of checkpointing.

13 Deliverables and Submission

13.1 What to Submit

A single zip archive containing:

- The complete source tree of the server (`kvdb-server`), the client (`kvdb-cli`), and the chaos test driver.
- A `Makefile` (or `CMakeLists.txt`) that builds everything with one command on a recent Ubuntu LTS using only the standard C/C++ toolchain.
- A `README.md` with build instructions, supported commands, and any known limitations.
- A `design.pdf` of at most 12 pages describing: the six-component architecture; the lock manager including the wait queue, fairness rule, upgrade rule, and deadlock detection; the two isolation levels and the lock-holding rules for each; the WAL format and the group commit protocol with the exact ordering rule; the recovery algorithm including how interleaved transaction records are handled; and a detailed walkthrough of every chaos scenario including the observed behavior plus the benchmark results.
- A `chaos/` directory with the chaos test driver and the logs from your final successful run.
- A `benchmark/` directory with the benchmark and your results.
- A `tests/` directory with unit tests for: the WAL serializer/deserializer, the recovery algorithm fed hand-crafted log files, the lock manager (compatibility, FIFO fairness, upgrade rules, timeout deadlock), and the end-to-end concurrent crash-recovery cycle.

13.2 Archive Naming

`Group[Number]_Project02_WAL.zip`, for example `Group17_Project02_WAL.zip`.

13.3 Demonstration

Each group will present a 25-minute live demonstration during the week following the submission deadline. It consists of three parts. First, a walkthrough of the lock manager and the group commit protocol using the design document. Second, a live execution of the chaos test against a freshly built server, with the instructor free to propose additional concurrent scenarios on the spot ("two clients race to update the same key, kill the server during the second client's commit, what is the recovered state?"). Third, a short oral examination in which each group member will be asked to explain, without notes, a specific component, with particular focus on the COMMIT ordering with group commit (why the pending buffer must not be applied before the fsync), and on why FIFO fairness in the wait queue is necessary to prevent writer starvation. All group members must be present.

14 Bonus Opportunities

Each independent. Up to 20 percentage points total (higher than other projects because of the increased base difficulty).

14.1 Wait-For Graph Deadlock Detection (up to 5%)

Replace the timeout-based deadlock detection with explicit wait-for graph cycle detection. Maintain a graph of "transaction A is waiting for a lock held by transaction B." Whenever an edge is added, run a cycle detection (DFS) starting from the new edge; if a cycle is found, abort the youngest transaction in the cycle. This produces faster, more precise deadlock resolution than the timeout approach. Document the algorithm and demonstrate it with a chaos scenario that triggers a 4-cycle deadlock and resolves it within milliseconds.

14.2 Snapshot Isolation (up to 10%)

Add a third isolation level, `SNAPSHOT`, that does not use locks for reads at all. Each transaction sees a consistent snapshot of the database as of the moment it began. Implement multi-version concurrency control (MVCC): each key carries a small version chain of (value, txn_id, commit_lsn) tuples; readers walk the chain to find the most recent version that committed before their own start LSN. Writes still use exclusive locks. This is roughly how PostgreSQL works, and it dramatically improves throughput on read-heavy workloads. Your design document must explain the MVCC implementation and your chaos test must include a write skew scenario showing that snapshot isolation does *not* provide full serializability (write skew is the canonical anomaly).

14.3 Replicated WAL with Sync Follower (up to 5%)

Add a `-follower host:port` option that streams the WAL to a single follower process. The leader does not acknowledge a commit until the follower has fsync'd the corresponding records. On leader crash, the follower has every acknowledged commit. (No automatic failover; restart the leader manually.) Demonstrate with a chaos scenario that kills the leader after acknowledged commits and verifies the follower has them.

15 Constraints and Prohibitions

The Following Will Result in Loss of Credit

- Use of any existing database, key-value store, or storage engine as a library. Includes SQLite, LevelDB, RocksDB, LMDB, BoltDB, BerkeleyDB, and bindings.
- Use of any existing lock manager, transaction manager, or concurrency control library. The lock manager must be your own code.
- Use of any existing binary serialization library (Protobuf, Cap'n Proto, FlatBuffers, MessagePack) for log or data records.
- Use of text-based persistence (JSON, CSV, XML) for log or data files.
- Skipping `fsync` on commit and calling the result durable. The group commit thread must call `fsync` (not `fflush`) between writing commit records and signaling waiting committers.
- Applying a transaction's pending buffer to the store before the group commit fsync that covers its commit record has completed. This is the canonical correctness bug for group commit and the chaos test will exercise it.
- Using a single global mutex across the entire server (instead of having the lock manager arbitrate at key-level granularity). The point of the project is concurrency; serializing every operation through one mutex defeats it.
- Use of Python at any layer, including the chaos test driver and the benchmark. Tests and scripts must be in C, C++, shell, or JavaScript.
- Submitting code whose commit history shows large, infrequent commits. Histories with three commits totaling thousands of lines will be scrutinized for academic integrity.

16 Required Reading

1. Sookocheff, K. "Write-ahead logging and the ARIES crash recovery algorithm," 2021. A blog post that explains WAL and crash recovery at the right level of abstraction. Read before Phase 2.
2. Eswaran, K. P., Gray, J. N., Lorie, R. A., and Traiger, I. L. "The Notions of Consistency and Predicate Locks in a Database System." *Communications of the ACM*, 19(11), 1976. The paper that introduced two-phase locking and serializability. Required reading before Phase 1; Sections 1–3 are the relevant ones.
3. Ramakrishnan, R. and Gehrke, J. *Database Management Systems*, 3rd edition, Chapter 16 ("Overview

of Transaction Management") and Chapter 17 ("Concurrency Control"). The reference for everything in Sections 5 and 6 of this manual. Read 16.1, 16.2, 17.1, 17.2 before Phase 1.

4. DeWitt, D. et al. "Implementation Techniques for Main Memory Database Systems." *ACM SIGMOD*, 1984. The paper that popularized group commit. Section 3 describes the technique exactly as you will implement it. Read before Phase 2.
5. Berenson, H. et al. "A Critique of ANSI SQL Isolation Levels." *ACM SIGMOD*, 1995. The paper that defined the modern understanding of isolation level anomalies (dirty read, non-repeatable read, write skew). Reference for Section 6 of this manual; relevant if you attempt the snapshot isolation bonus.
6. Pavlo, A. *CMU 15-445 Database Systems*. Lectures 15–17 cover concurrency control and lectures 21–22 cover logging and recovery. Freely available on the CMU course website.

Penalty Structure

- Missing documentation: –10%
- Security vulnerabilities: –5% per issue
- Poor code quality: –5% to –15%
- Academic integrity violations: Zero grade

Contact & Support

Teaching Assistants:
Jalal Ahmed: `bscs23134@itu.edu.pk`
Khadijah Farooqi: `bscs23128@itu.edu.pk`

Office Hours: Monday & Thursday,
11:30 AM – 2:30 PM